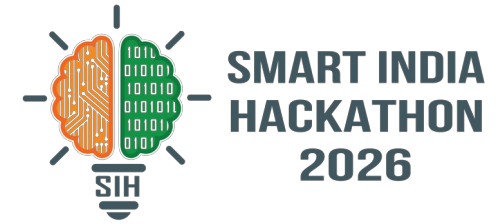


SMART INDIA HACKATHON 2026



TITLE PAGE

Problem Statement ID – SIH26159

Problem Statement Title – SecureMailScope: AI-Assisted Cryptographic Security

Posture Assessment for Secure Email Communications

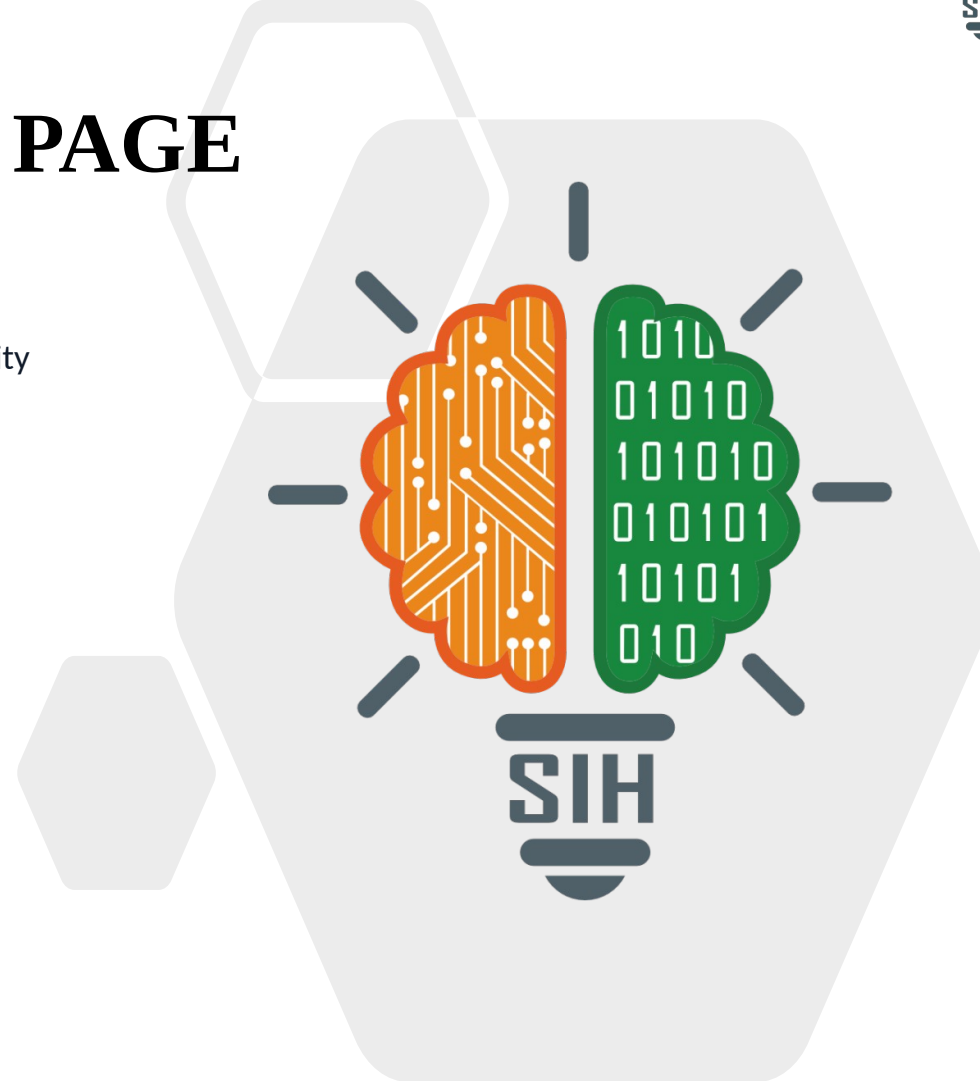
Organisation – National Technical Research Organisation (NTRO)

Theme – Blockchain & Cybersecurity

PS Category – Software

Team ID – 146876

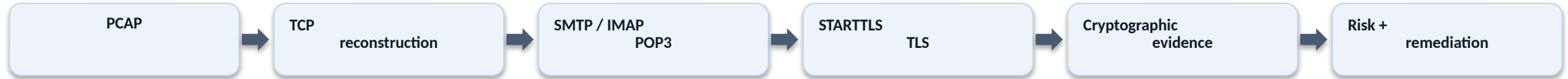
Team Name (Registered on portal) – Zero-Day



IDEA TITLE

SecureMailScope turns passive email packet captures into evidence-backed cryptographic investigations.

Every conclusion is traceable to the packets that establish it — and where a capture cannot show something, it says so.



The investigation overview, on 9 synthetic captures. Actual product output.

What it does

Reads authorized PCAP/PCAPNG, reconstructs TCP sessions, identifies the email protocol from the dialogue rather than the port, reconstructs the TLS handshake, extracts certificate evidence where it is observable, applies 25 policy rules and scores the result.

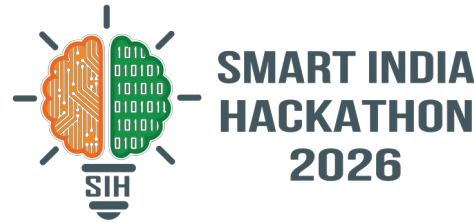
How it addresses SIH26159

The organisation already holds the captures. No probe, no scan, no credential and no change to a production mail server — which also means an authorised investigator with no access to the host can still assess how it protected traffic.

What makes it different

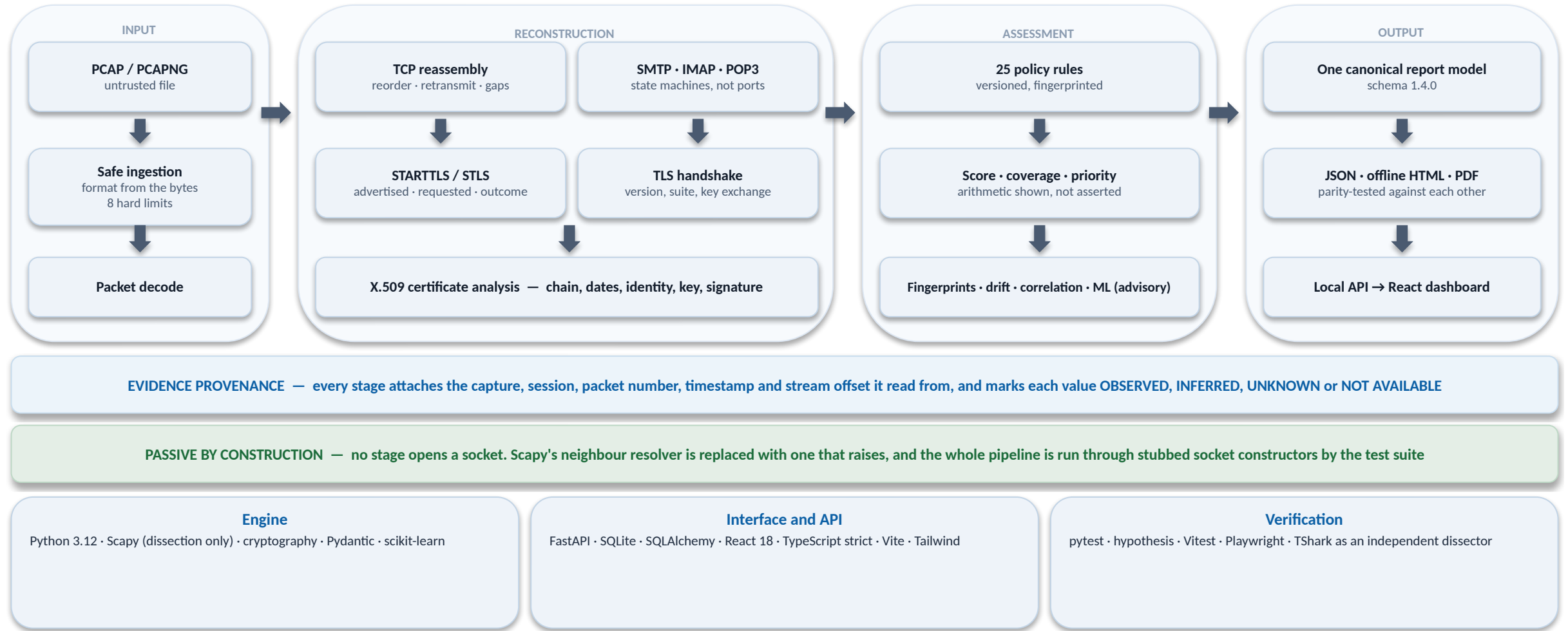
Packet-level provenance on every finding · cryptographic DNA and drift across captures · cross-session correlation · explainable scoring with the arithmetic shown · fully local and offline · stated limits instead of silent gaps

TECHNICAL APPROACH



Zero-Day

One direction of flow. Each stage reads only what the previous one produced, and attaches the packets it read it from.



FEASIBILITY AND VIABILITY

Feasible because it is built and measured. Every figure below is from the release commit, re-run — not an estimate.

1,357

backend tests pass

1,367

with the TShark cross-check

89

frontend tests

5

browser end-to-end specs

156

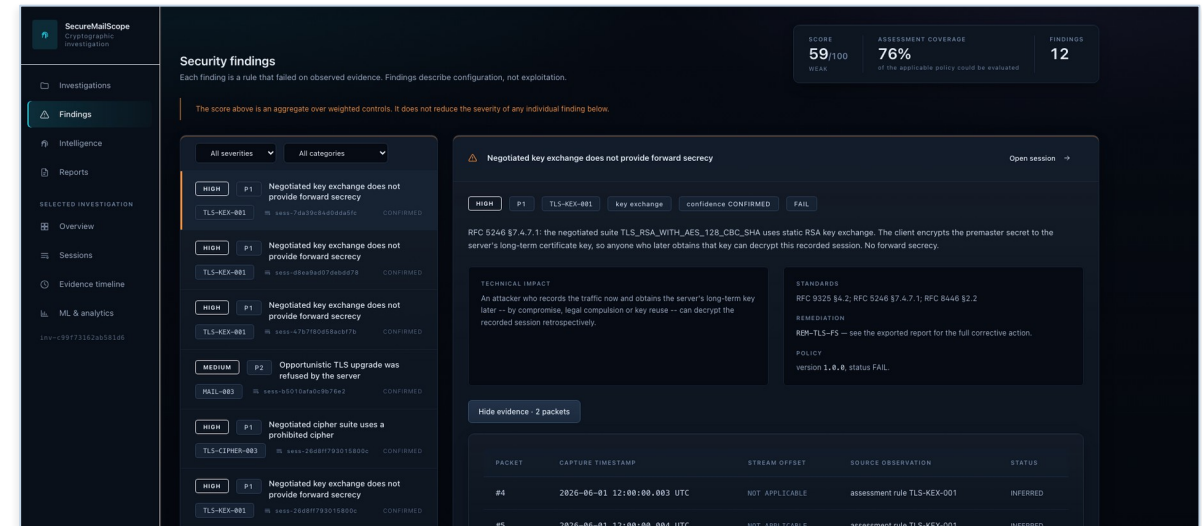
files ruff + mpy clean

0

known dependency vulnerabilities

How a finding is proved

1. A rule fails on a reconstructed observation — here, static RSA key exchange, so no forward secrecy.
 2. The engine attaches the packets it evaluated: #4 and #5, with their capture timestamps and stream offsets.
 3. The capture and session ids are printed beside them, so an analyst can open the same packets in Wireshark and check.
 4. Packet metadata only. Reconstructed payload bytes are never included in a report or shown in the interface.
- Expectations are hand-derived and committed as manifests while the captures are generated, so a test cannot confirm its own output; ten TShark cross-checks compare our dissection against Wireshark's, so a bug in our parser cannot validate itself.



SecureMailScope
Investigation

Security findings
Each finding is a rule that failed on observed evidence. Findings describe configuration, not exploitation.

The score above is an aggregate over weighted controls. It does not reduce the severity of any individual finding below.

Score: 59/100
Remediation Coverage: 76%
Findings: 12

Negotiated key exchange does not provide forward secrecy
Open session →

Technical Impact:
RFC 5246 §7.4.7.1: the negotiated suite TLS_RSA_WITH_AES_128_CBC_SHA uses static RSA key exchange. The client encrypts the premaster secret to the server's long-term certificate key, so anyone who later obtains that key can decrypt this recorded session. No forward secrecy.

Standards:
RFC 5246 §4.2; RFC 5246 §7.4.7.1; RFC 8446 §2.2

Remediation:
REM-TLS-P5 — see the exported report for the full corrective action.

Policy:
version 1.0.0, status FAIL

Hide evidence - 2 packets

Packet	Capture Timestamp	Stream Offset	Source Observation	Status
#4	2026-06-01 12:00:00.003 UTC	NOT APPLICABLE	assessment rule TLS-KEY-001	INFERRED
#5	2026-06-01 12:00:00.004 UTC	NOT APPLICABLE	assessment rule TLS-KEY-001	INFERRED

The finding, and the packets it was evaluated against. Actual product output.

Limits, stated not hidden

TLS 1.3 encrypts the Certificate message, so a passive capture cannot expose it — reported NOT_AVAILABLE with the reason, never blank and never guessed. TLS 1.2 chains are read and verified in full. The supervised classifier is NOT VALIDATED for real-world use: it was measured on synthetic servers only, it drives no finding and no score, and the interface says so on screen.

IMPACT AND BENEFITS

For authorised email-security investigation. Each visual below is the product output that supports the benefit beside it.

DISCOVER

The cryptographic configuration actually negotiated — version, cipher suite, key exchange, certificate — read from traffic the organisation already holds, without touching the host.

EXPLAIN

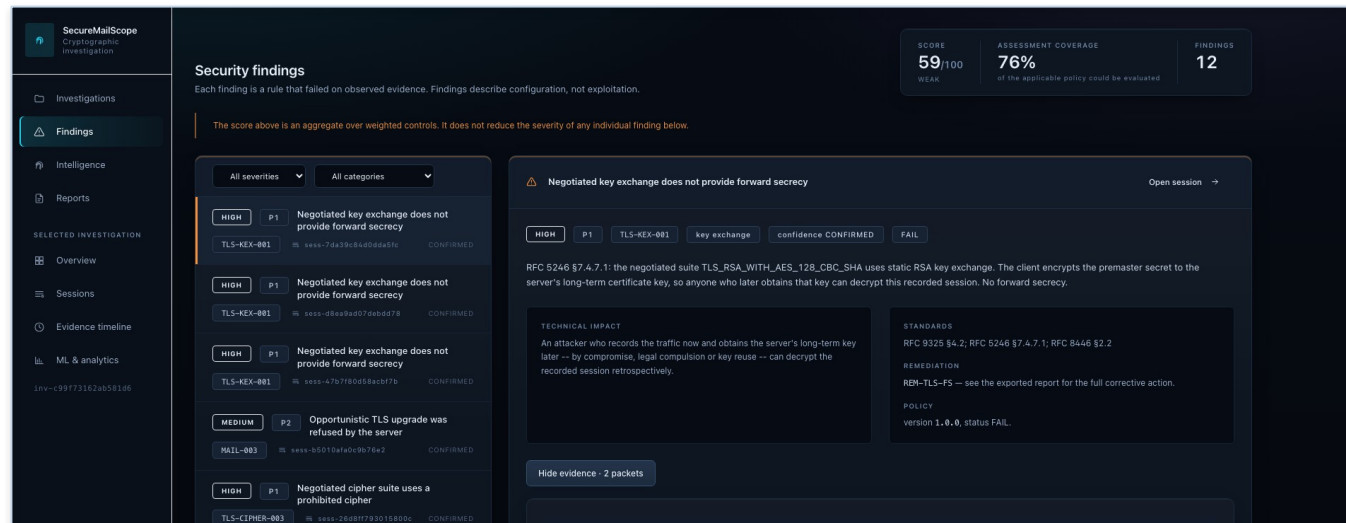
Every finding names the rule it failed, the RFC clause it applies and the packets that establish it, so the conclusion can be checked rather than believed.

PRIORITIZE

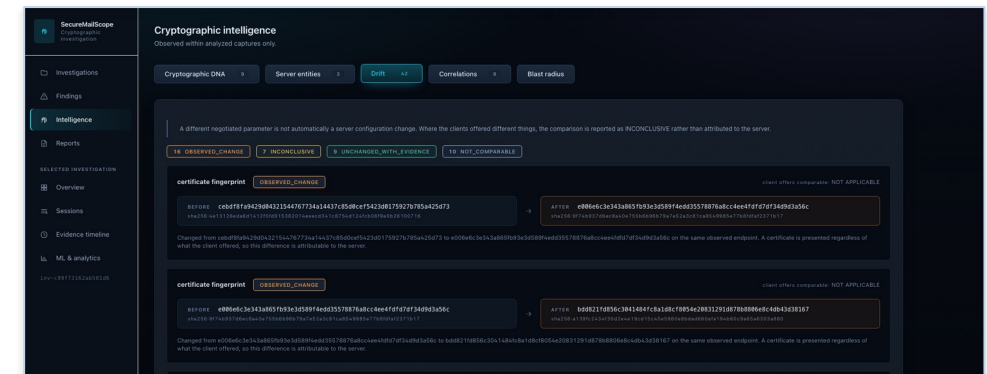
Severity and confidence produce a ranked priority with a named remediation, so 'what do I fix first' has an answer and a reason.

TRACK

Drift compares one endpoint across captures. Where the clients asked different questions the comparison is reported INCONCLUSIVE rather than blamed on the server.



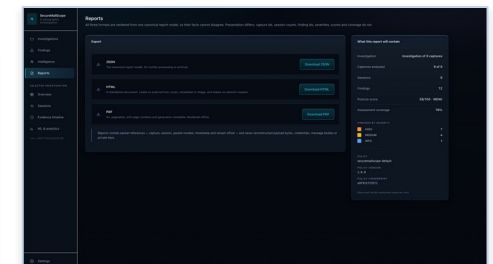
One finding, its rule, its remediation and the packets behind it.



Configuration drift, before and after.

REPORT

One canonical model produces JSON, a self-contained offline HTML document and a PDF, so their facts cannot disagree.



Export, in three formats.

Benefits are demonstrated on synthetic captures. No deployment, adoption figure or real-world detection rate is claimed.

The standards the rules cite and the parsers implement, and how every claim in this deck is checked.

Core TLS and PKI

RFC 8446 — TLS 1.3
RFC 5246 — TLS 1.2
RFC 9325 — secure use of TLS (2022)
RFC 8996 — deprecating TLS 1.0 and 1.1
RFC 7457 — known attacks on TLS
RFC 4492 — ECC cipher suites
RFC 5280 — X.509 certificate and CRL profile
RFC 6125 — service identity verification
NIST SP 800-52 Rev. 2 · IANA TLS registry

Email protocols

RFC 5321 — SMTP
RFC 3207 — SMTP over TLS (STARTTLS)
RFC 9051 — IMAP 4rev2
RFC 1939 — POP3
RFC 2595 — TLS with IMAP, POP3 and ACAP
RFC 2606 — reserved names (.invalid, used by every fixture)

Validation tools

Wireshark / TShark — independent dissection
pytest · hypothesis — property and regression tests
Playwright · Vitest — browser and unit tests
pip-audit · npm audit · CycloneDX — supply chain
ruff · mypy — lint and strict typing

HOW EVERY CLAIM IN THIS DECK IS CHECKED

Deterministic fixtures

generated from a fixed seed; identical bytes on every machine

Hand-derived manifests

expectations written by hand and committed, not read back

Engine under test

1,357 tests; the captures themselves stay out of git

Independent dissector

ten TShark cross-checks against Wireshark's own parse

Real browser, real backend

five Playwright specs, including a restart for persistence

Report parity

JSON, HTML and PDF asserted to agree, field by field

Repository and demonstration

Repository <https://github.com/sgtsuijith141-wg/securemailscope>

Demonstration *Demo video included in submission package.*